String (TH)
Monday, March 23, 2026 9:34 AM

Tuple

↳ ข้อมูลชนิด list แต่ immutable → ไม่สามารถแก้ไขได้

Syntax: (item1, item2, item3)

* immutable → ไม่สามารถ อัพเดต
หรือ เพิ่ม/ลบ ของ ต่างๆ เช่น
- x[index] = new_value
- .append, .insert, .remove, .sort

ตัวอย่าง:
```
tup = (10, 20, 30)
print(tup[0])  → 10
for i in range(len(tup)):
    print(i, tup[i])
```
0 10
1 20
2 30

. Function แปลง เป็น list หรือ tuple

1) list(x) → แปลง x เป็น list
2) tuple(x) → แปลง x เป็น tuple

เช่น range() ได้

```
my_list = [1,2,3]
my_tuple = tuple(my_list)
print(my_list)  → [1, 2, 3]
print(my_tuple) → (1, 2, 3)
my_range = list(range(0,5))   [0,1,2,3,4]
print(my_range) → [0, 1, 2, 3, 4]
```

String

- ข้อความ → ตัวอักษร หลายๆ ตัว
↳ a sequence of characters.

```
my_string = "ABCDEF"
```

↳ a sequence of characters.

```
my_string = "ABCDEF"
     index   0 1 2 3 4 5

print(my_string[0]) → A
```

String คือ a sequence of characters and it's immutable and you can index.

string ≈ tuple แต่ใช้สำหรับตัวอักษร

Indexing

```
       -11-10-9-8-7 -6 -5-4-3-2-1
my_str = "Hello World"
          0 1 2 3 4 5 6 7 8 9 10

print(my_str[4]) → o
print(my_str[8]) → r
print(my_str[-1]) → d
```

ทำไม่ได้: ~~my_str[0] = 'A'~~ ← update ไม่ได้ เพราะ immutable!

Slicing

↳ คือ การ เอาของออกมาตาม index หลายตัว (sequence of index)

↳ 2 Steps:

1) บอกว่า index อะไรบ้าง
2) ไปเอามา ทีละ index!

syntax: string[start : end : step]

1) start:end:step → ขอ sequence ของ index เหมือน range
2) หลังจากได้ indexes ก็ไปเอาของออกมา

0 1 2 3 4 5 6 7 ..

2) หลังจากได้ indexes แล้วจึงเอาของออกมา

```
          0 1 2 3 4 5 6 7
my_str = "A B C D E F G H"
new_str = my_str[1:4:1]      ① [1, 2, 3]   ② "BCD"
print(new_str)  → BCD

your_str = my_str[2:7:2]     ① [2, 4, 6]   ② "CEG" ← ได้เป็น string ใหม่ออกมา
CEG ← print(your_str)
```

Default values ของ slicing

→ No step : x[start:end:] , x[start:end]
  ↳ default step = 1

→ No end : x[start::step]
  ↳ default end = เอาจนถึงตัวสุดท้าย
  * ถ้าเป็น negative index → ตัวแรกสุด

→ No start : x[:stop:step]
  ↳ default start = คือตัวแรกสุด

→ ผสมได้   x[start:] , x[start::]
  (default end กับ step)

→ x[::] → เอาทุกตัว จากตัวแรกถึงตัวสุดท้าย
  x[::-1] → ? → reverse string!
  ↑ default start, end

EX:
```
y = "ABCD"
x = y[::-1]   → "DCBA"
print(x)      → DCBA
```

✱ Slicing ใช้กับ list, tuple, string

Ex
```
my_list = [1, 2, 5, 7]
new_list = my_list[0:2]
```

Ex
```
my_list = [1, 2, 3, 4]
new_list = my_list[0:2]
```
[0, 1] → [1, 2]
index
new_list

Ex
```
full_names = "Patty_Lynn_Smith"
```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

```
first  = full_names[0:5]   # [:5] → Patty
second = full_names[6:10]  → Lynn
third  = full_names[11:]   → Smith
```

Ex2
```
pattern = "A|B|C"
first = pattern[0:1]   # pattern[0]
second = pattern[2]
```

For loop

↳ for loop ตามจำนวนตัวอักษร → "ABCD"

A, B, C, D

```
name = "Alice"
for char in name:
    print(char)
```
0 1 2 3 4

A
l
i
c
e

Ex ตัวอย่าง ตัวอักษร
```
target = "t"
my_str = "Hello There TtT"
count = 0
for char in my_str:
    if char == target:
        count += 1
print(count)
```
→ 2

* len() function ใช้กับ string ได้!
↳ ตัวอย่างจำนวน ตัวอักษร

* len() (funct

↳ นับจำนวนตัวอักษร

```
my_str = "Alice"
print(len(my_str))  →  5

for i in range(len(my_str)):
    print(i, my_str[i])  →  0 A
                            1 l
                            2 i
                            3 c
                            4 e
```

String Operators

- \+ = เอา string มาต่อกัน : "AB" + "CD" → "ABCD" (เป็น string อันใหม่!)
- *n = เอา string ต่อกันเป็น n ครั้ง : "AB" * 3 ↳ "ABABAB"
- in = หาว่ามี string อยู่ในอีก string ไหม? → True, False

```
("AB" in "ABCDE") → True
("ab" in "ABCDE") → False
```

String Methods

Syntax: string.method_name()

↳ จะส่ง string หรือข้อมูลใหม่ออกมา!

↳ ต้องเอาไปเก็บไว้ หรือใช้งานทันที!!

Method set 1 : string testing method

↳ จะ return True หรือ False

Method set 1: String Testing Method

↳ จะ return True หรือ False

- is lower() : True ถ้าทุกตัวอักษรเป็น lower case
  ↳ Ex: "abc".islower() → True
  "Abc".islower() → False
- is upper() : True ถ้าทุกตัวอักษรเป็น upper case
- is space() : True ถ้า " — " ช่องว่าง (tab, newline)
- is alnum() : True ถ้า " — " alphabet หรือ ตัวเลข
- is alpha() : True ถ้า " — " alphabet
- is digit() : True ถ้า " — " ตัวเลข

```
x = "ABC123"
print(x.isdigit()) → False
print(x[3:].isdigit()) → True
```

3,4,5 → "123".isdigit()

String Method set #2 : สร้างใหม่

- .lower() → ส่ง string ใหม่ที่เป็น lower_case ทั้งหมด
- .upper() → ส่ง string ใหม่ที่เป็น upper_case ทั้งหมด

Ex

```
my_str = "Alice"
print(my_str.lower()) → alice
                        "alice"
name = my_str.upper()
print(my_str, name) → Alice ALICE
name = name.lower() ←
name = name.upper() ←
```

```
name = name.lower()  ←
name = name.upper()  ←
print(name)  → ALICE
```

"Chain string methods → จากซ้ายไปขวา.

```
print("AbCd".lower().upper().lower())
      "abcd".upper().lower()
             "ABCD".lower() → "abcd"
```

---

strip() family methods : ลบ ช่องว่าง ออกจาก string ที่อยู่ด้านหน้า หรือด้านหลัง

- rstrip() → ลบ ช่องว่าง ที่อยู่ด้านขวา/หลัง ออก ทั้งหมด
- lstrip() → ลบ ช่องว่าง ที่อยู่ ด้านหน้า ออก ทั้งหมด
- strip() → ลบ ช่องว่าง ที่อยู่ ทั้ง ด้านหน้าและ ด้านหลัง ออก ทั้งหมด!

Ex

```
x = "__abcd__"
print(x.rstrip(), "a")    __abcd a
print(x.lstrip(), "a")    abcd   a
print(x.strip(), "a")     abcd a
```

- strip ลบ ตัวอักษร
  - ↳ lstrip("a"), rstrip("x"), strip("z")
  - ↳ logic เหมือนเดิม ลบ ตัวนั้น ทั้งหมด ที่อยู่ ด้านหน้าหรือ หลัง

Ex

```
x = "aaababccbcc"
print(x.lstrip('a'))  → "babccbcc"
print(x.rstrip('c'))  → "aaababccb"
```

print(x.lstrip('a')) → 'bab bccbcc'
print(x.rstrip('c')) → 'aaababbccb'

print(x.strip('a').rstrip('c').upper())

"baabccbcc".rstrip('c')

"baabccb".upper()

↳ BAABCCB

String Method #3 : ค้นหา

.find(str)

↳ คือ index แรก ของ str ใน string หลัก

↳ ถ้าหาไม่เจอจะคืน -1 ออกมา

"ABCDEF".find("DE") → 3

"AB CD".find("z") → -1

.replace(old_str, new_str)

↳ แทน old_str ด้วย new_str ใน string หลัก ของเรา ทุกที่

↳ return a new string

X = "ABCABD ABJ"

print(X.replace("BC", "X")) → AXABDABJ

print(X.replace("AB", "")) → CDJ

print

print(x.replace("AB", "")) → (CD)

(empty)

.split(str) method

สำคัญมาก

↳ จะตัด string ตาม str ที่ให้ ออกมา เป็น **list of strings**

Ex1

"Hello How are you?".split(" ")

["Hello", "How", "are", "you?"]

Ex2

"133//334//756".split("//")

["133", "334", "756"]

* note default ของ split() คือช่องว่าง

เช่น "A B C D".split() → ['A', 'B', 'C', 'D']

Use case หลัก : ใช้ร่วมกับ input()!

↳ input() ส่ง string กับมาเสมอ!

↳ สมมติ User พิมพ์ตัวเลข หลายตัว คั่นด้วย comma เข้ามา

เช่น "10, 20, 30, 40"

แล้วเราต้องการตัวเลข แต่ละตัวไปใช้?

↓ เปลี่ยนให้เป็น list ของตัวเลข

[10, 20, 30, 40]

[10, 20, 30, 40]

1. รับตัวเลขหลายตัว คั่นด้วย comma จาก user
2. split string ที่ comma → list of string
3. เปลี่ยน string แต่ละตัวใน list ให้เป็น int

"10, 20, 30"

```
numbers = input("Enter numbers: ")
list_string = numbers.split(",")        -> ["10", "20", "30"]
for i in range(len(list_string)):
    list_string[i] = int(list_string[i])
print(list_string)                      -> [10, 20, 30]
print(sum(list_string))                 -> 60
```

.join(lst) method

↳ เปลี่ยน lst ของ string ให้เป็น string
โดยที่จะต่อ ของแต่ละตัว ด้วย string หลัก

```
",".join(["A", "B", "C"])  -> "A,B,C"
```